# MatchWatch

USER STORIES DELIVERABLE

Pedro Fernandes

202304500

UC Engenharia de Software

# 0 Contents

# 1 Friends

This section includes a description of the befriending feature alongside its mockups and acceptance tests.

## 1.1 Statement

> ***As a*** *Movie Lover*
>
> ***I want to*** *send, receive, and manage friend requests within the app*
>
> ***So that*** *I can see what they are watching, what they are looking forward to watch, share reviews and invite them to watch parties*

## 1.2 Business Value

Friends are the trigger for watch parties which drives adoption of a core feature. Seeing friends' reviews incentivizes the creation of content and more meaningful content overall. Since there is a constant update of what one watches, their reviews and what they are interested in, there is always a reason to come back.

## 1.3 Dependencies and Assumptions

- Users must have an account before they send, receive, and manage friend requests.
- Users must be identifiable with a unique key.
- User profiles must contain their reviews and watchlist as it is a core purpose of the befriending feature.
- Backend must support friend-request state management: `pending`, `accepted`, `declined`.
- There is a tab for managing these requests.

## 1.4 Scenarios

### Normal Scenario: Sending and Accepting a Friend Request

**Description**

A user searches for another user by username, sends a friend request, and the recipient accepts it. Both users can then view each other's personal profile and send invitations for a watch party.

## Mockup

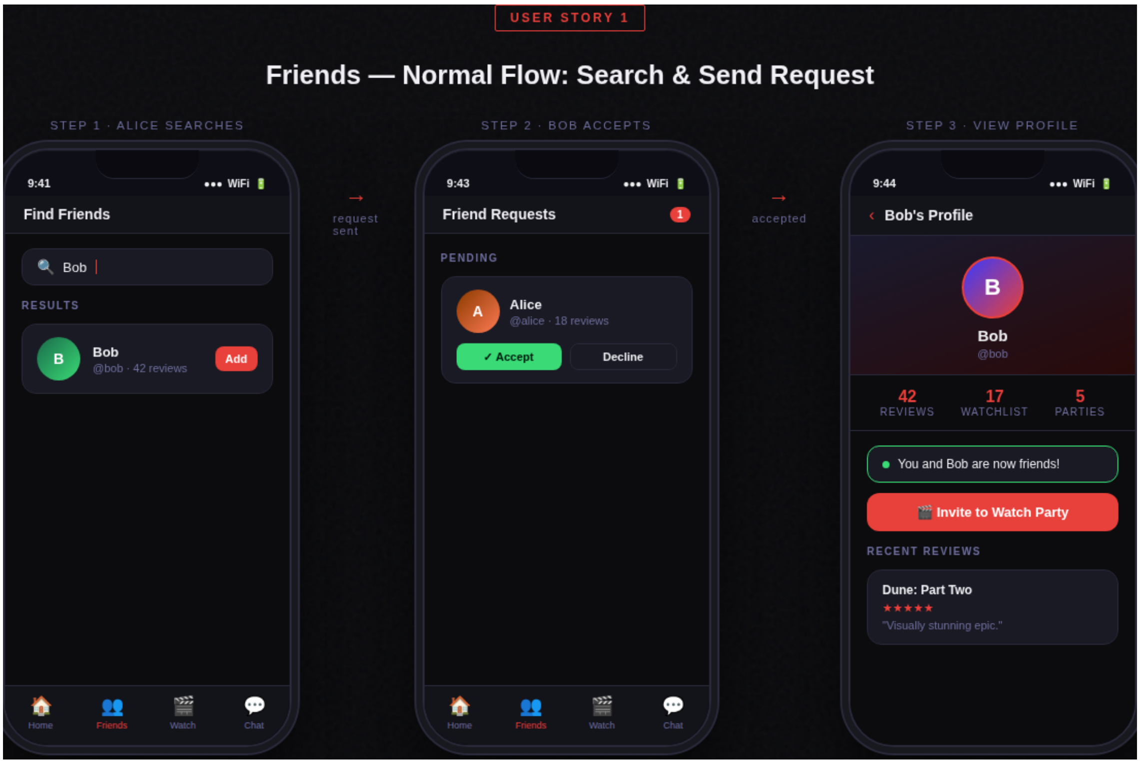


Figure 1: Friends — Normal Flow: Search & Send Request

## Acceptance Test

```
Feature: Friend Requests

  Scenario: Sending and accepting a friend request
    Given a registered user "Alice" is logged into
       MatchWatch
    And another registered user "Bob" exists in the
       system
    When Alice searches for the username "Bob"
    And Alice sends a friend request
    And Bob logs into MatchWatch
    And Bob accepts the friend request
    Then Alice and Bob should appear in each other's
       friend list
    And both users should be able to view each other's
       profiles
```

## Exceptional Scenario: Sending a Request to a Non-existent User

## Description

A user types a username that does not exist in the system and attempts to send a friend request. The app prevents the action and informs the user clearly.

### Mockup

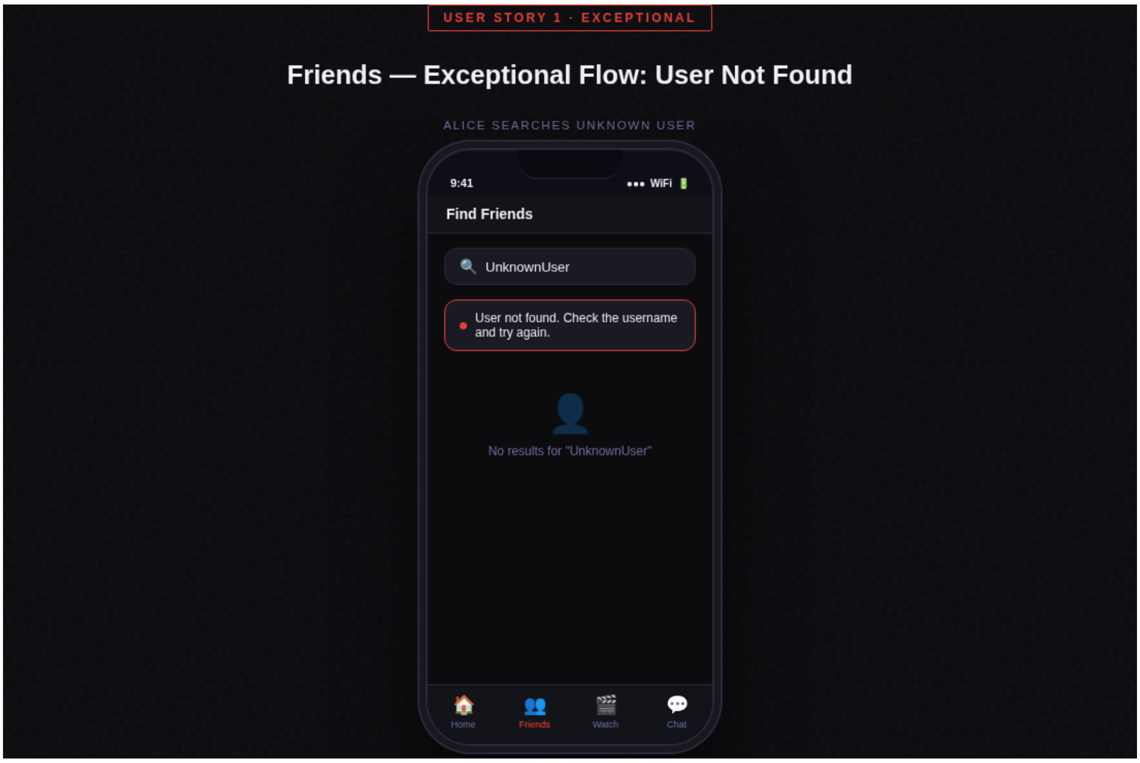


Figure 2: Friends — Exceptional Flow: User Not Found

### Acceptance Test

```
Feature: Friend Requests

  Scenario: Sending a friend request to a non-existent
     user
    Given a registered user "Alice" is logged into
       MatchWatch
    When Alice searches for the username "UnknownUser"
    And Alice attempts to send a friend request
    Then the system should display the message "User not
       found"
    And no friend request should be created
```

# 2 Chat Functionality

This section includes a description of the in-app chat feature alongside its mockups and acceptance tests.

## 2.1 Statement

> ***As a*** *Movie Lover*
>
> ***I want to*** *send and receive messages with my friends inside the app*
>
> ***So that*** *I can discuss movies, share recommendations, and coordinate watch parties without leaving MatchWatch*

## 2.2 Business Value

In-app chat removes the need to switch to an external messaging platform, keeping users inside MatchWatch longer. The chat feature functions as the coordination channel for watch parties, making it a dependency for that feature's success. Active conversations provide a natural trigger for re-opening the app through push notifications.

## 2.3 Dependencies and Assumptions

- You can only message someone if you're already friends with them.
- Chat history is saved server-side, so conversations don't disappear between sessions.
- There needs to be a dedicated tab in the app where users can find and manage their chats.

## 2.4 Scenarios

### Normal Scenario: Sending and Receiving a Message

#### Description

A user opens a conversation with a friend, types a message about a movie they just watched, and the friend receives it in real time. The friend replies and both messages appear in the conversation thread.

#### Mockup

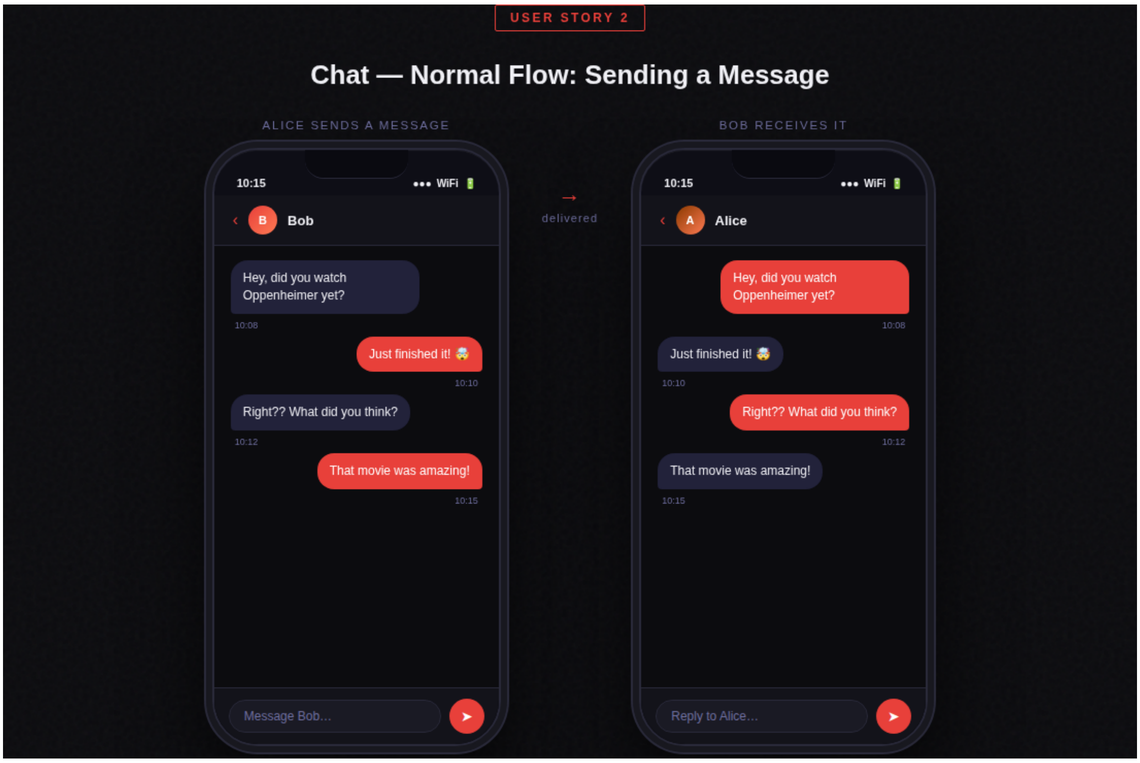


Figure 3: Chat — Normal Flow: Sending a Message

### Acceptance Test

```
Feature: Chat Messaging

  Scenario: Sending and receiving a message between
     friends
    Given Alice and Bob are friends on MatchWatch
    And Alice is logged into the application
    When Alice opens a chat with Bob
    And Alice sends the message "That movie was amazing!
       "
    Then Bob should receive the message
    And the message should appear in the conversation
       thread
```

## Exceptional Scenario: Message Fails to Send (No Connection)

### Description

A user attempts to send a message while offline or with a very poor connection. The message fails to deliver and the app informs the user that something went wrong.

### Mockup

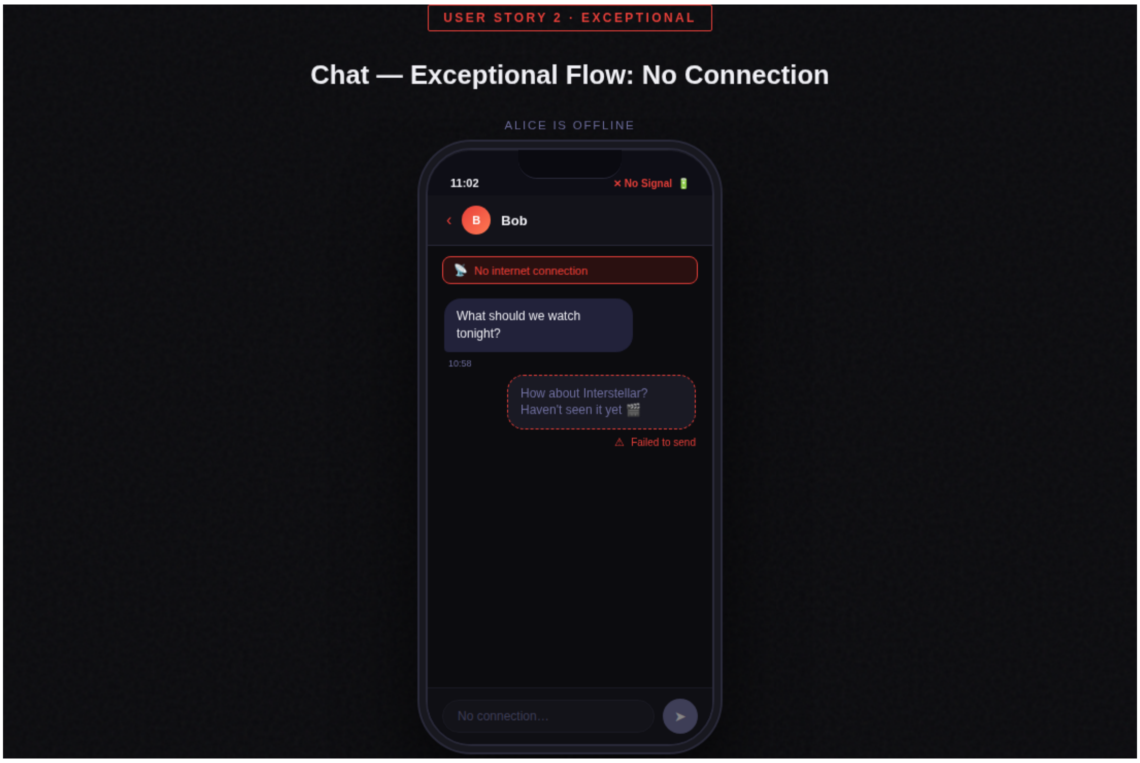


Figure 4: Chat — Exceptional Flow: No Connection

### Acceptance Test

```
Feature: Chat Messaging

  Scenario: Message fails to send due to no internet
     connection
    Given Alice and Bob are friends on MatchWatch
    And Alice has no internet connection
    When Alice attempts to send the message "Let's watch
        a movie tonight"
    Then the system should notify Alice that the message
        failed to send
    And the message should be queued for automatic retry
```

### AI Usage Disclosure

Throughout this project I made use of AI tools in a few specific areas, which I want to be upfront about.

- This document was originally written in Markdown. I used Claude to help convert and format it into LaTeX.
- The UI mockups started as an HTML file that Claude helped generate. I then went in and adjusted the design to better match my vision for the app.
- The **Acceptance Tests** were written with some AI assistance for the initial structure and wording, which I then reviewed and adapted to fit the actual scenarios.

Everything in this document has been read through and is submitted as my own work.